CONCEPTS & DATA MODEL

Jira Assets, attribute types, and why some block Cloud migration

Background a reader needs before touching the remediation: what Assets is, how schemas, object types, attributes, objects and values relate, and exactly why the Confluence (and Version/Project) attribute types stop a JCMA migration.

Assets DC→Cloud migration · attribute remediation series · document 1 of 4

Document set: (1) Data Model & Concepts · (2) Process & Stages · (3) Scripts Reference · (4) Runbook, Recovery & Rollback. Each part is standalone and can be regenerated independently.

1. What this is about

Jira **Assets** (called **Insight** before the rebrand) is the CMDB built into Jira Service Management. It stores structured records — servers, applications, licences, documentation links and so on. When a Jira Data Center (DC) instance is migrated to Cloud with the **Jira Cloud Migration Assistant (JCMA)**, most Assets data moves across, but a few **special attribute types** have no equivalent in Cloud. If any object actually holds a value in such an attribute, JCMA refuses to continue and lists the unsupported attribute types as errors. This series explains how to find those blockers, back up the data, clear them so migration proceeds, and rebuild the information in Cloud.

> **Reader assumption.** You do not need prior Assets knowledge. You do need access to the Jira DC and Cloud sandboxes, and the ability to create API tokens. All actions below are first rehearsed on a sandbox, never on production.

2. The Assets data model

Five entities matter. Understanding how they nest is the whole key to the remediation, because the problem lives in one specific place — the attribute *definition* — while the data you must preserve lives in another — the attribute *value*.

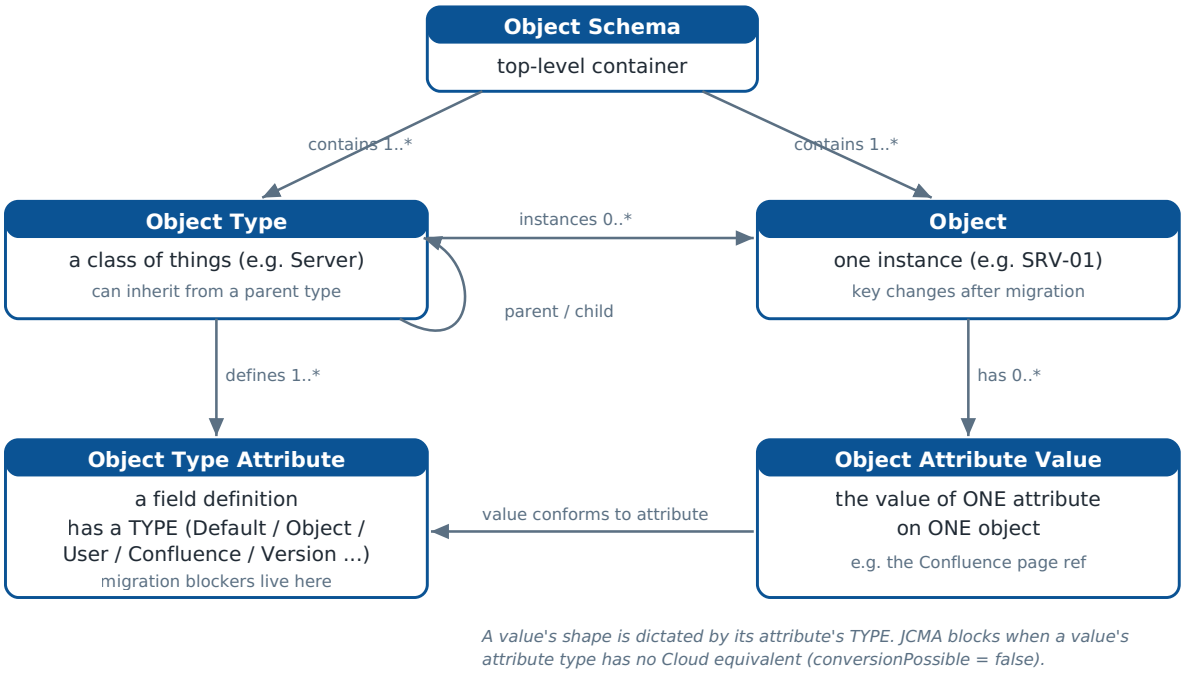

Figure 1. Assets data model and the relationships between its entities.

- **Object Schema** — the top-level container (e.g. an "IT" schema). Everything else lives inside a schema. A schema has a numeric id; you supply that id to every script.
- **Object Type** — a class of things within the schema (e.g. "Server", "Application"). Object types can form a hierarchy: a child type **inherits** the attributes of its parent. This inheritance is central later.
- **Object Type Attribute** — a field *definition* on an object type (e.g. "Documentation"). Crucially, each attribute has a **type** (a number 0–7) that fixes what kind of value it holds. **The migration blocker is a property of this definition.**
- **Object** — one concrete instance of an object type (e.g. server "SRV-01"). Each object has a **key** like `CMDB-3493`. Keys are regenerated in Cloud, so you cannot rely on them to line objects up across the migration.
- **Object Attribute Value** — the value of *one* attribute on *one* object (e.g. the Confluence page that SRV-01's "Documentation" attribute points to). This is the data you back up and later re-apply.

> **The one sentence to remember.** A value's shape is dictated by its attribute's **type**. JCMA blocks when a populated value belongs to an attribute whose type has no Cloud equivalent. The AQL response flags exactly this with `"conversionPossible": false`.

3. Attribute types — and which ones block

Every attribute carries an integer `type`. Type **0 (Default)** additionally carries a `defaultType` sub-id that picks Text, URL, Textarea, Integer, Date, etc. Default attributes migrate cleanly.

The non-default types are "special" — they reference DC-specific entities through application links.

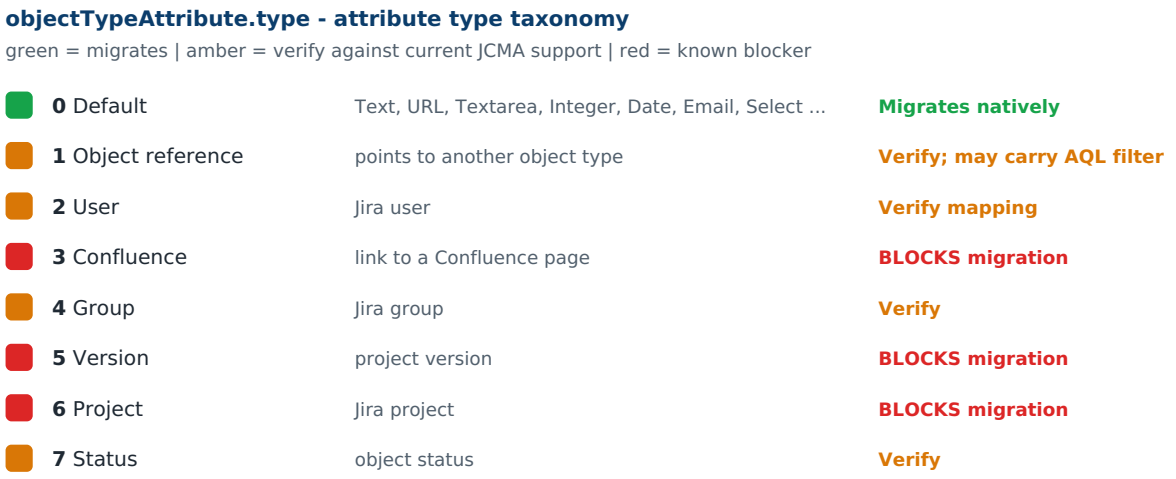

Figure 2. Attribute type taxonomy. Red types are the known migration blockers.

The **Confluence (3)**, **Version (5)** and **Project (6)** types are the ones that stop JCMA. They point at things — a Confluence page on a linked instance, a project version, a Jira project — that are addressed by DC-internal identifiers and application links with no like-for-like Cloud counterpart. The remaining non-default types (Object reference, User, Group, Status) should be **verified** against Atlassian's current JCMA support matrix rather than assumed safe, because that matrix changes over time.

> **Confluence type has a second twist.** Its value is a Confluence *page id*. Even where you could keep a link, Confluence page ids change when Confluence itself migrates DC→Cloud (via CCMA), so the stored reference must be **re-mapped**, not merely copied. And Assets *Cloud* has no Confluence attribute type at all — the replacement lands as a plain **URL** or **Text** attribute.

4. Jira / JCMA limitations and caveats

Limitation	Consequence for remediation
You cannot change an attribute's **type** in place in DC.	There is no "convert Confluence→Text" button. You create a new attribute of the wanted type and migrate the data into it, or rebuild on the Cloud side.
Deleting an attribute is **destructive and irreversible**.	Deleting cascades to all its values with no undo (only a database restore). Always back up first; prefer clearing values over deleting the attribute.
The type field is **locked while values exist**.	To change a type you must first clear or delete the values — another reason clearing is the primary lever.
Object **keys change** in Cloud.	Re-import / re-apply must match on a stable identifier (Name, or a captured mapping), never the old key.

Confluence **page ids change** DC→Cloud.	A DC→Cloud page-id map (by space + title) is required before any value can be re-applied in Cloud.
Assets **Cloud has no Confluence type**.	Replacement attribute is URL or Text; you lose the rich page picker but keep a working link.
Inheritance.	A blocker declared on a parent type is inherited by every child; remediating one type is not enough.
AQL usage.	An Object-reference attribute's IQL filter may reference a blocker by name; removing the blocker can break that filter.

5. Derived / dependent attributes — can they cause extra errors?

Assets has **no calculated or formula attributes**, so no attribute is ever computed *from* the Confluence attribute. There is therefore no hidden "derived type" that silently breaks. However, three real dependency vectors can produce *additional* migration errors and must be checked before remediating:

1. **Inheritance** — the same blocker attribute appearing on a parent and its children (same `objectTypeAttributeId`).
2. **Other independent blockers** — Version and Project attributes elsewhere in the schema that block on their own merits, even after Confluence is fixed.
3. **AQL usage** — an Object-reference attribute whose filter text references a blocker attribute by name.

The discovery script (Script 5, see document 3) maps all three across a schema so remediation is complete rather than piecemeal. Run it first.